

5. Interface Segregation Principle (ISP) in Java

5.1 Introduction

The **Interface Segregation Principle (ISP)** is the fourth **SOLID** principle. It aims to keep interfaces **small and specific**, avoiding fat interfaces that force implementing classes to define unused methods.

5.2 Problem It Solves

Avoids polluting classes with irrelevant behaviors by breaking large interfaces into smaller ones.

5.3 Definition of ISP

"Clients should not be forced to depend on methods they do not use."

5.4 Real-World Analogy

Imagine a multifunction remote that includes buttons for air conditioner, TV, fan, lights. If you're only using the TV, the rest of the buttons are just clutter — better to have separate remotes (interfaces).

5.5 Symptoms of ISP Violation

- Classes with unused method implementations
- Interfaces with too many responsibilities
- Widespread usage of `UnsupportedOperationException`

5.6 Benefits of ISP

- More maintainable code
- Encourages cohesive interface design
- Easier to implement and test
- Less breakage when interfaces evolve

5.7 Implementation Guidelines

- Split large interfaces into smaller ones
- Prefer composition over bloated inheritance
- Group related functionality logically

5.8 Examples

Example1: Violation: Printer Interface

```
package com.mannemlokeshev.violation.example1;

interface MultifunctionPrinter {
    void print(String content);

    void scan(String content);

    void fax(String content);
}
```

```
}  
  
class OldPrinter implements MultiFunctionPrinter {  
    public void print(String content) {  
        System.out.println("Printing: " + content);  
    }  
  
    public void scan(String content) {  
        throw new UnsupportedOperationException("Scan not supported");  
    }  
  
    public void fax(String content) {  
        throw new UnsupportedOperationException("Fax not supported");  
    }  
}  
  
public class PrinterClient {  
    public static void main(String[] args) {  
        // ...  
    }  
}
```

Example1: Compliant : Printer Interface

```
package com.mannemlokes.compliant.example1;  
  
interface Printer {  
    void print(String content);  
}  
  
interface Scanner {  
    void scan(String content);  
}  
  
interface Fax {  
    void fax(String content);  
}  
  
class BasicPrinter implements Printer {  
    public void print(String content) {  
        System.out.println("Printing: " + content);  
    }  
}  
  
public class PrinterClient {  
    public static void main(String[] args) {  
        Printer printer = new BasicPrinter();  
        printer.print("Hello ISP!");  
    }  
}
```

Output:

```
Printing: Hello ISP!
```

Example2: Violation: Worker Types

```
package com.mannemlokesh.violation.example2;

interface Worker {
    void work();

    void eat();
}

class Robot implements Worker {
    public void work() {
        System.out.println("Robot working");
    }

    public void eat() {
        throw new UnsupportedOperationException("Robot can't eat");
    }
}

public class WorkerClient {

    public static void main(String[] args) {
        // ...
    }
}
```

Example2: Compliant : Worker Types

```
package com.mannemlokesh.compliant.example2;

interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

class Human implements Workable, Eatable {
    public void work() {
        System.out.println("Human working");
    }

    public void eat() {
        System.out.println("Human eating");
    }
}

class Robot implements Workable {
    public void work() {
        System.out.println("Robot working");
    }
}

public class WorkerClient {
    public static void main(String[] args) {
        Workable human = new Human();
        Workable robot = new Robot();
    }
}
```

```
        human.work();
        robot.work();

        Eatable eater = new Human();
        eater.eat();
    }
}
```

Output:

```
Human working
Robot working
Human eating
```

Example3: Complaint : Device Interface Decomposition

```
package com.mannemlokesh.compliant.example3;

interface Switchable {
    void turnOn();

    void turnOff();
}

interface Dimmable {
    void dim(int level);
}

interface ColorChangable {
    void changeColor(String color);
}

class SmartLight implements Switchable, Dimmable, ColorChangable {
    public void turnOn() {
        System.out.println("Light turned ON");
    }

    public void turnOff() {
        System.out.println("Light turned OFF");
    }

    public void dim(int level) {
        System.out.println("Dimming to level: " + level);
    }

    public void changeColor(String color) {
        System.out.println("Color changed to: " + color);
    }
}

public class DeviceClient {
    public static void main(String[] args) {
        SmartLight light = new SmartLight();
        light.turnOn();
        light.dim(50);
        light.changeColor("Blue");
        light.turnOff();
    }
}
```

```
}  
}
```

Output:

```
Light turned ON  
Dimming to level: 50  
Color changed to: Blue  
Light turned OFF
```

5.9 Common Pitfalls

- Using "God interfaces" with 10+ methods
- Ignoring the fact that many classes won't need all interface methods
- Tight coupling to interfaces that evolve often

5.10 ISP vs SRP vs DIP

Principle	Focus
ISP	Clients shouldn't depend on unused APIs
SRP	Class should have one reason to change
DIP	Depend on abstraction, not concretions

5.11 When to Apply ISP

- When you notice UnsupportedOperationException
- When many classes implement only a part of the interface
- During interface design/refactoring

5.12 Summary

Feature	Description
Intent	Split large interfaces into smaller ones
Use Case	Device control, plugins, handler logic
Key Benefit	Decouples logic, simplifies implementation

The Interface Segregation Principle promotes modular, clean, and flexible design. It protects clients from being forced to depend on methods they don't need.